



**James' Super Cool  
IS 201 Cheat Sheet**

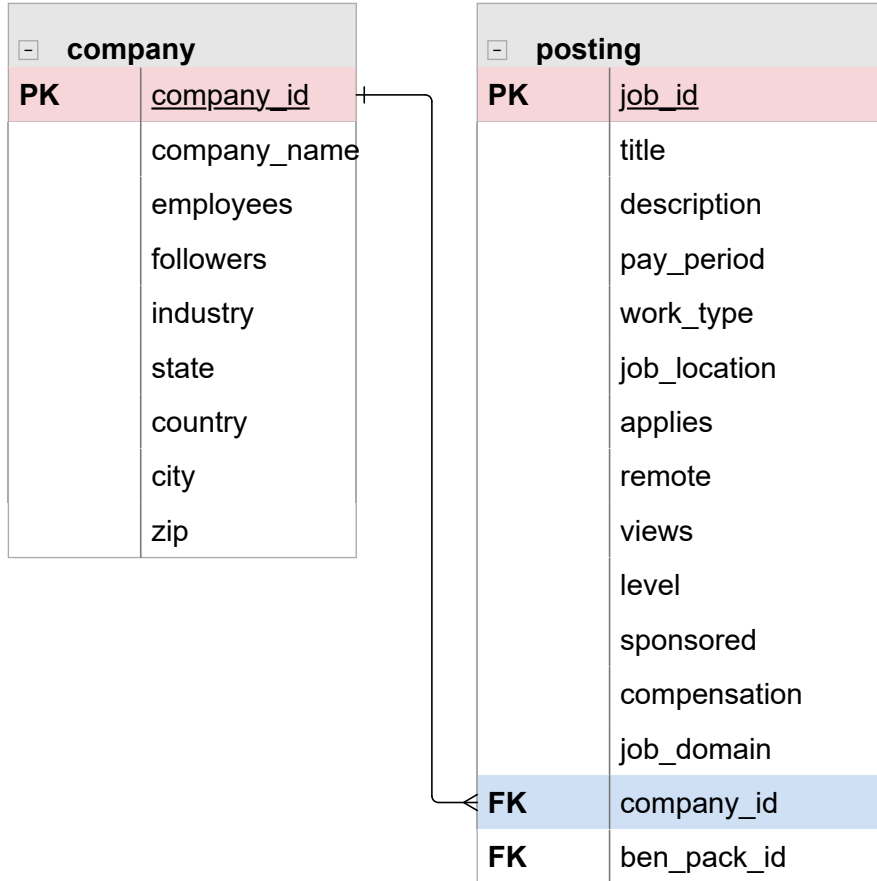
**You**



# Index

| Slide | Topic                              |
|-------|------------------------------------|
| 2     | <a href="#"><u>ERDs</u></a>        |
| 6     | <a href="#"><u>SQL</u></a>         |
| 8     | <a href="#"><u>Flow Charts</u></a> |
| 9     | <a href="#"><u>VBA</u></a>         |
| 15    | <a href="#"><u>Statistics</u></a>  |
| 16    | <a href="#"><u>Tableau</u></a>     |
| 17    | <a href="#"><u>Solver</u></a>      |
| 18    | <a href="#"><u>HTML</u></a>        |
| 27    | <a href="#"><u>CSS</u></a>         |

# ERD: Primary / Foreign Keys

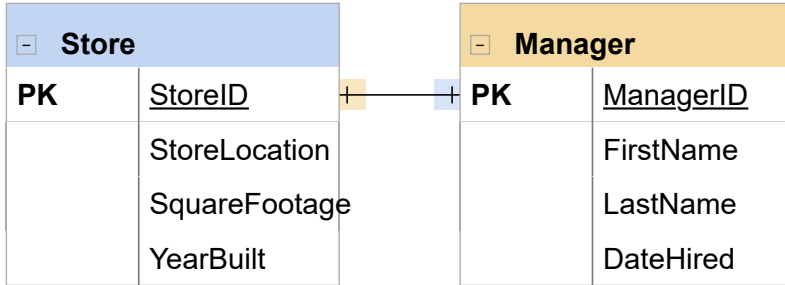


A **primary key** is a unique identifier (think social security number)

A **foreign key** is another table's primary key, used to link two tables together

**Entity**  
**Relationship**  
**Diagram**

# ERD: Cardinality

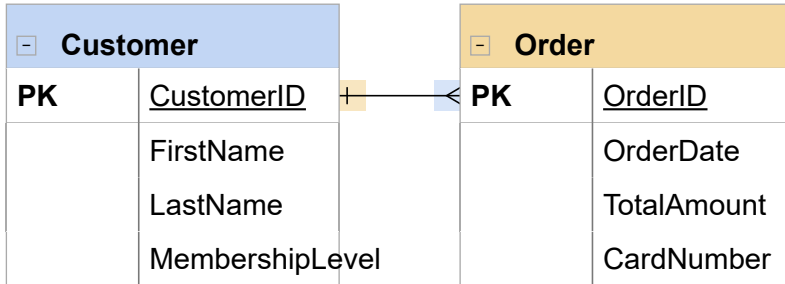


Read left-to-right AND right-to-left

## One to One (1:1)

A store has one manager

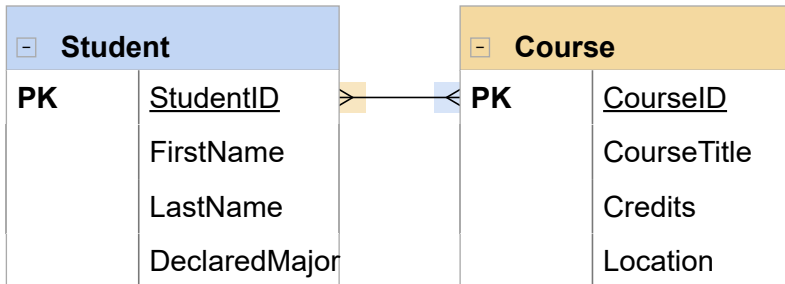
A manager works at one store



## One to Many (1:N)

A customer can have multiple orders

An order belongs to one customer



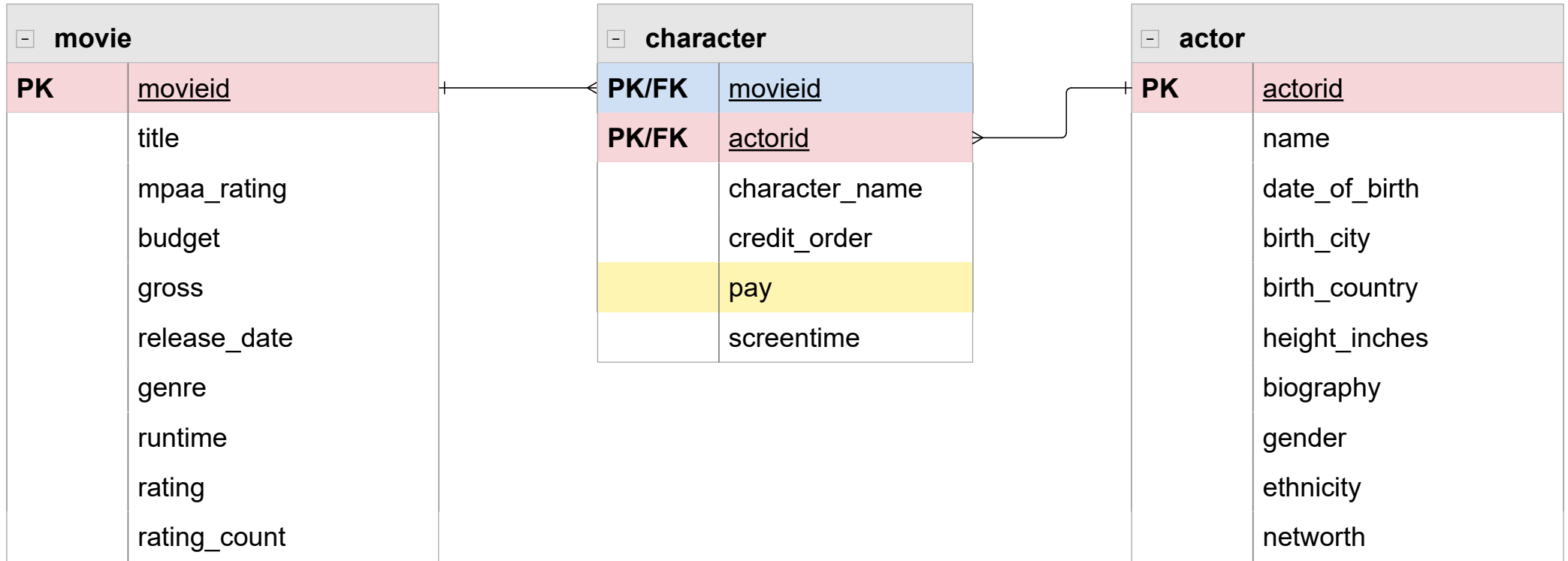
## Many to Many (M:N)

A student can enroll in many courses

A course can have lots of students

*Note! M:N cardinality requires a composite table (next slide)*

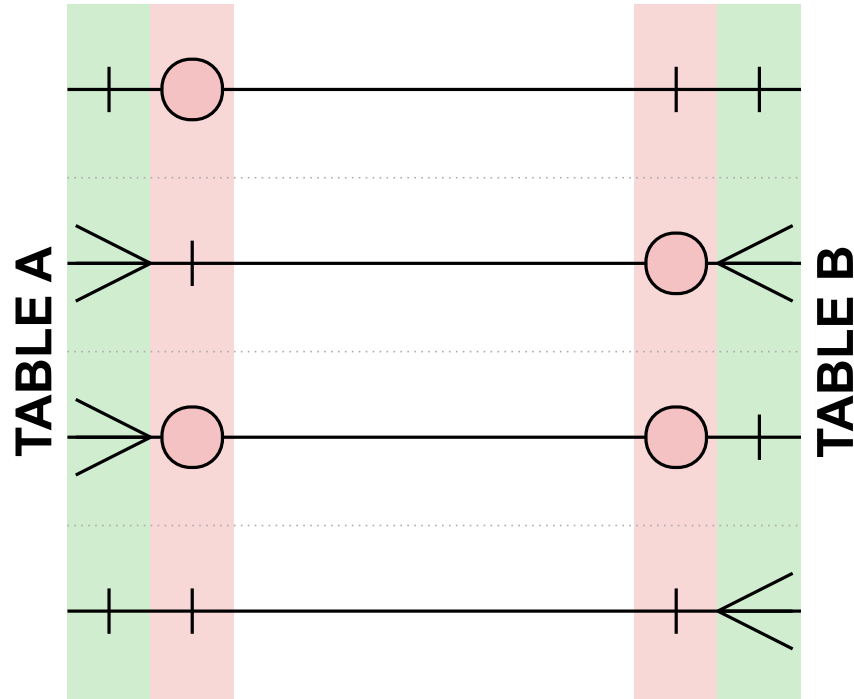
# ERD: Composite Table



To know how much money Tom Hanks got **paid** to play Woody in *Toy Story*, you need to know both the **movie** and the **actor**. His pay is probably different than when he was Woody in *Toy Story 2*, or when he was Forrest Gump in *Forrest Gump*

# ERD: Minimum / Maximum Cardinality

The inner marks are minimum (0 or 1)  
and the outer marks are maximum (1 or many)



A could have 1 B or 1 B

B could have 0 A or 1 A

A could have 0 B or many B

B could have 1 A or many A

A could have 0 B or 1 B

B could have 0 A or many A

A could have 1 B or many B

B could have 1 A or 1 A

*Think: a student could have 0 cars, but they could also have multiple.*

# SQL

```
1 SELECT      -- attributes -- do SELECT DISTINCT to only show unique results
2 FROM        -- tableA
3 JOIN        -- tableB ON tableA.attribute = tableB.attribute (order doesn't matter)
4 WHERE       -- attribute filters [=, !=, <>, IS, LIKE '%__%', IN ("__","__","__")]
5     AND/OR  -- you only say WHERE once, but can have many filters
6 GROUP BY    -- all non-aggregated attributes when aggregating
7 HAVING      -- group-based filters (not used in the SQL project)
8 ORDER BY    -- attributes in ASC (default) / DESC order
9 LIMIT       -- to xx top results
10
11 -- aggregate(attribute) AS newName
12 count()
13 avg()
14 min()
15 max()
16 -- etc.
```

Structured  
Query  
Language

## SQL: Where Use Cases

ta\_name = "James"

→ Exact match

ta\_name != "James"  
ta\_name <> "James"

→ Not exact match  
*(these are equivalent)*

ta\_name **IS** NULL

→ NULL means blank data  
(can also do **IS NOT** NULL)

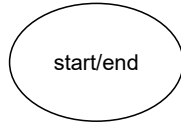
ta\_name **LIKE** '%ame%'

→ The text ("*ame*") is contained within the attribute  
(% = *wildcard*)

ta\_name **IN** ("James", "Robert", "Frankie")

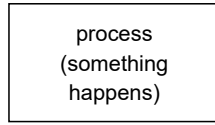
→ Exact match for any of these

# Flow Charts

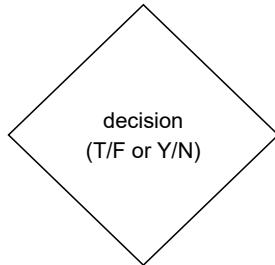


start/end

you can only have ONE start  
but you CAN have multiple  
ends

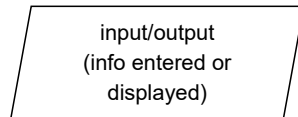


process  
(something  
happens)

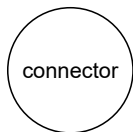


decision  
(T/F or Y/N)

decisions are the ONLY thing  
that can have more than one  
arrow pointing OUT of them



input/output  
(info entered or  
displayed)

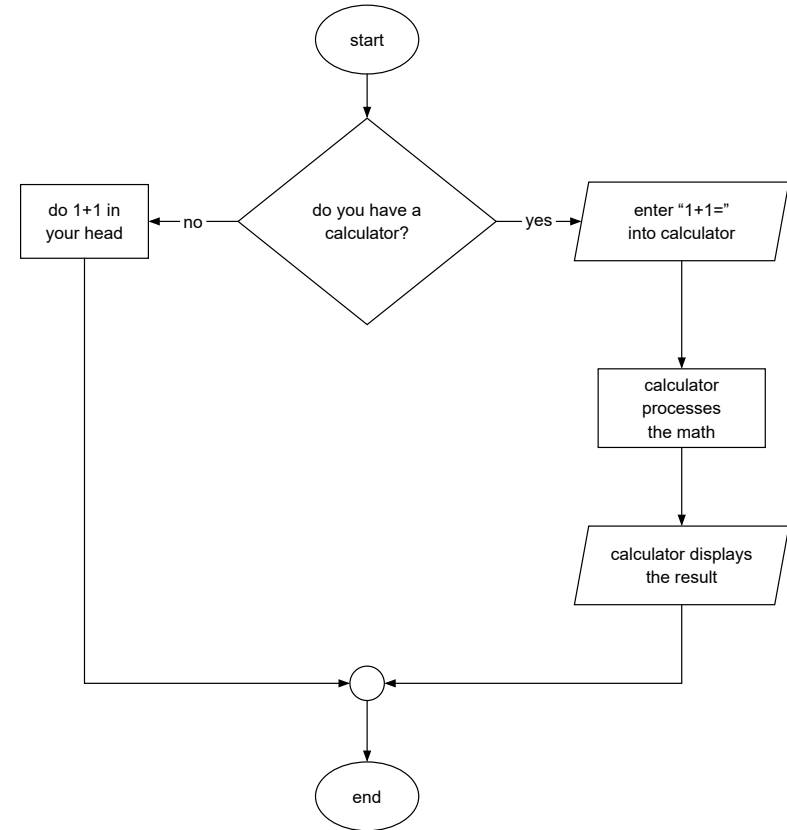


connector

connectors are the ONLY  
thing that can have more than  
one arrow pointing INTO them

file > export as > pdf

## EXAMPLE: How to solve 1+1



# VBA: Basics

Option Explicit 'This makes it so that you can only use variables you've declared (VERY RECOMMENDED)

Sub thisIsMySubName()

    'BASICS!

    'Generally you will first refer to an object type (like a Sheet)

    'and then tell it what you want to do (like delete or add or copy, etc):

        ' Sheets and Worksheets are the same thing

        Sheets("MySheet").Delete

        Sheets.Add.name = "MySheet"

        Worksheets("MySheet").Activate

        ' Range and Cells are SIMILAR but different: Range("E7") = Cells(7, 5)

        Range("A1").Activate

        ActiveCell.Value = "Hello"

        Range("A1:D3").Copy

        Range("E5").PasteSpecial

        ' Columns and Rows are what they sound like

        Columns("B:D").Delete

Visual  
Basic for  
Applications

# VBA: Navigation and Misc.

'NAVIGATION!

```
'
ActiveCell.End (xlDown)
ActiveCell.End (xlToLeft)

' Move (rows, columns) based on a cell or range
ActiveCell.Offset(1, 2).Value = "NewSpot"
    Range("A1").Offset(3, 4).Value = "NewSpot" 'The cell that now says "NewSpot" is E4
```

' MISCELLANEOUS!

```
' Bolding
Range("A1").Font.Bold = True

' Autofitting columns
Columns.AutoFit 'all columns
Columns("A:C").AutoFit 'specific columns

' Currency format
Range("A1").Style = "Currency"

' Bottom border
Range("A1").Borders(xlEdgeBottom).LineStyle = xlContinuous

' Text splicing
name = "James"
first_initial = Left(name, 1)
```

# VBA: Variables

```
'DECLARE VARIABLES!
Dim i As Integer    'a whole number
Dim j As Double     'a number with a decimal
Dim k As String     'a line of text
Dim l As Boolean    'true or false
Dim m As Worksheet

i = 5
j = 1.2345
k = "Hello there! General Kenobi!"
l = True
Set m = Sheets("Sheet1") 'objects (sheets, books, ranges) use Set instead of =

'USER INTERFACE!

'Tell the user something
MsgBox "Oh hello there user, welcome to my spreadsheet"

'Ask for an input
Dim username As String
username = InputBox("What's your name?")

' COMBINING STRINGS

Dim myOutput As String
Dim name As String
name = "James"

' COMbine strings with ampersands (&)-- don't forget spaces
myOutput = "Hello. My name is " & name & " and this is Disney Channel!"
```

## 'CONDITIONALS!

```
' If Statements
If i < 0 Then
    ' your code here
ElseIf i < 5 Then
    ' your code here
ElseIf i < 10 Then
    ' your code here
Else
    ' your code here
End If
```

```
'Case Statements
Select Case fruit
    Case "apple", "strawberry"
        Color = "red"
    Case "grape"
        Color = "purple"
    Case Else
        Color = "unknown"
End Select
```

```
'Another Case Statement
Select Case temperature
    Case Is >= 100
        feel = "too hot"
    Case 80 To 100
        feel = "pretty warm"
    Case Is < 80
        feel = "I'm probably staying out of the pool"
End Select
```

# VBA: Conditionals

'LOOPS!

# VBA: Loops

```
'For Loop
For i = 0 To 5
    ' your code here
Next i
```

```
'For Each Loop
Dim x As Range
For Each x In Range("A2", Cells(lastrow, "A"))
    ' your code here
Next
```

```
'For Each Loop 2
Dim x As Range
Dim mtRange As Range
Set myRange = Range("A1:E5")
For Each x In myRange
    ' your code here
Next
```

```
'Do While Loop
Do While i = 5 'condition does NOT have to be numbers
    ' your code here
Loop
```

```
' Do Until Loop
Do Until i = 10 'condition does NOT have to be numbers
    'your code here
Loop
```

End Sub

# VBA: Functions

```
Function thisIsMyFunctionName(num As Integer, tf As Boolean) As String
```

```
' a function declaration is just like a set of dims  
' the INPUT is in the parentheses ()  
' the OUTPUT is outside of it  
' so this function KIND OF has this baked into it:
```

```
Dim num As Integer  
Dim tf As Boolean  
Dim thisIsMyFunctionName As String
```

```
' the INPUT then comes from your excel sheet  
' for example, in your sheet you cell might say:  
' =thisIsMyFunctionName(15, True)
```

```
' the OUTPUT comes from your function  
' set the function's name equal to something  
' in this case, our output has to be a string:
```

```
thisIsMyFunctionName = "That's a very nice hat you have there, partner!"
```

```
' generally your inputs will be used to calculate/decide your output
```

```
End Function
```

# Statistics

## TEST & USE CASE

### Correlation

numeric – numeric  
(Pearson  $r$  coefficient)

### Regression

numeric – multi. numeric

### ANOVA

numeric – mult. categorical

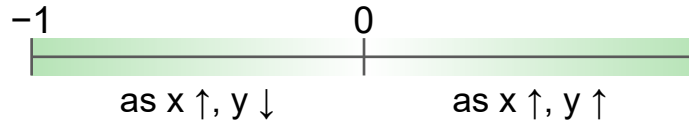
### T-test

numeric – 2 pair categorical

## OUTPUT

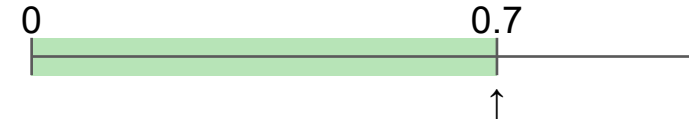
## HOW TO READ IT

→  $r$



*strength = distance from 0; sign = direction*

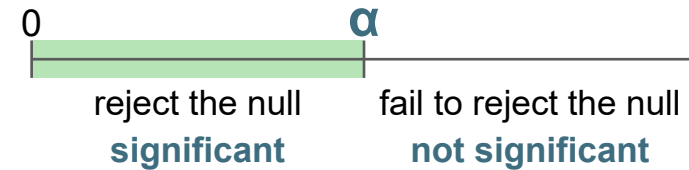
→  $R^2$



70% of the variance in  $y$   
is explained by the  $x$ 's

usually 0.05

→  $p$



*$p$  works like a percentage:  
0.05 = 5% to see this or more extreme*

## Null Hypothesis

$H_0$ : No  $x$ 's are  
significant

## Alt Hypothesis

$H_A$ : At least one  $x$  is  
significant

## Scientific Notation

$aE^b = a \times 10^b$

$aE3 = a \times 1000$  (big)

$aE-3 = a \times 0.001$   
(small)

## Tableau

- “Default” settings are usually a good starting point — check them first
- Switching the x- and y-axis can reveal a clearer story
- Watch your sort/ordering — Tableau doesn’t always order the way you expect

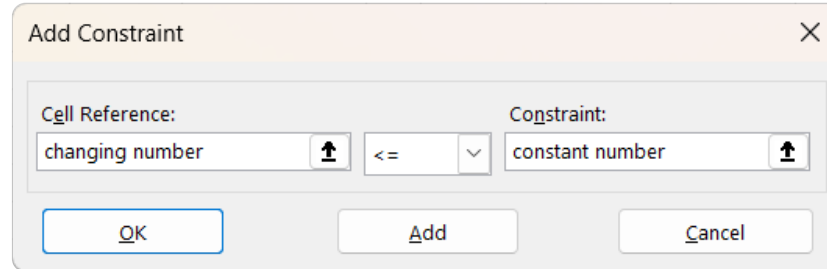
# Solver

## Two important Excel formulas:

=sum(A1:A5) →  $A1 + A2 + A3 + A4 + A5$

=sumproduct(A1:A5, B11:B15) →  $A1*B11 + A2*B12 + \dots + A5*B15$   
*the two ranges should be the same size and shape*

## Constraints:



Add Constraint

Cell Reference: changing number

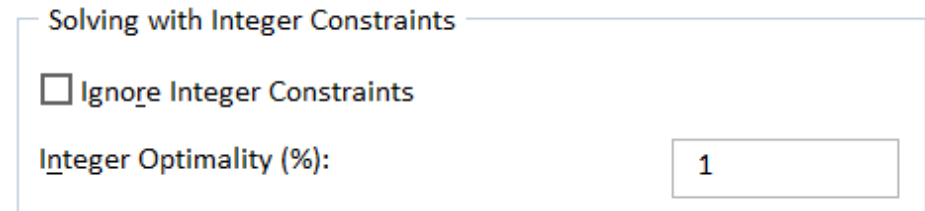
Constraint: constant number

<=

OK Add Cancel

## If integer isn't working:

Solver > Options > uncheck this:



Solving with Integer Constraints

☐ Ignore Integer Constraints

Integer Optimality (%): 1

# HTML: Setup

In VS Code, inside a `.html` file, type ! then Enter to generate:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-
6          scale=1.0">
7      <title>Document</title>
8  </head>
9  <body>
10 </body>
11 </html>
```



[startbootstrap.com/themes/portfolio-resume](https://startbootstrap.com/themes/portfolio-resume)

HyperText  
Markup  
Language

# HTML: Tags



```
1  <!-- This is a comment -->
2  <h1>Main heading</h1>
3  <h2>Section heading</h2>
4  <h3>Subsection</h3>
5  <p>This is a paragraph.</p>
6  <!-- Line break -->
7  <br>
8  <!-- Unordered list (bullets) -->
9  <ul>
10     <li>Item 1</li>
11     <li>Item 2</li>
12 </ul>
13 <!-- Ordered list (numbers) -->
14 <ol>
15     <li>Item 1</li>
16     <li>Item 2</li>
17 </ol>
```

*The start and end tags act like parentheses: <tag> stuff </tag>*

# HTML: Anchors



```
1 <!-- Hyperlinks -->
2 <a href="https://www.google.com">Visit Google</a>
3 <!-- Folder references -->
4 <a href="otherpage.html">Visit Other Page</a>
5 <a href="subfolder/page.html">Visit Subfolder Page</a>
6 <a href="../yetanotherpage.html">Visit yet another page</a>
7 <!-- On-page anchors (any tag can have an id, not just <a>) -->
8 <a id="section1">Section 1</a>
9 <a href="#section1">Go to Section 1</a>
```

*../ means go to the parent folder (or root directory)*

# HTML: Images



```
1 <!-- Images -->
2 
3 
```

*Notice: images do NOT have an end tag `</img>` (neither do line breaks `</br>`)*

# HTML: Divisions

Divs don't inherently do anything — they're invisible boxes you put content into so you can isolate it for positioning and CSS styling using classes.

## HTML

```
1 <div class="square-  
  image">  
2       
3 </div>
```



## CSS

```
1 .square-image {  
2     width: 200px;  
3     height: 200px;  
4 }
```



For more, see: **CSS: Selectors/Properties/Values**

# HTML: Embedding

*Sometimes doesn't work until you go live*

## YouTube

1. Click **Share**
2. Click **Embed**
3. Click **Copy**
4. Paste into your code

## Tableau

1. Click the share icon
2. Click **Copy Embed Code**
3. Paste into your code

## HTML: Folder Structure & Formatting

**Use folders!** Keep `css/`, `html/`, `images/`, and `js/` separate, with `index.html` at the project root.

To quickly format everything correctly (tabs, long lines, etc.) in VS Code:

Right click → Format Document

## HTML: Uploading to GitHub

1. Click **New** to create a repository
2. Click **Create repository**
3. On the repo page: **Add file** → **Upload files**
4. Click **Commit changes**

**Upload the CONTENTS of your folder, not the folder itself**

## Going Live

1. Go to **Settings**
2. Click **Pages** in the sidebar
3. Under Branch, select `main` (instead of `None`) and **Save**

After a couple of minutes, refresh the page — there will be a link to your website at the top.

# CSS: Basics

```
1 <!-- Connect CSS to HTML, in your <head> -->
2 <link rel="stylesheet" href="styles.css" />
```



```
1 selector {
2     property: value;
3     property2: value2;
4 }
```



CSS is **cascading** because more recent styling supersedes previous styling — if you set the font color to blue, then later set it to red, it will be red.

Cascading  
Style  
Sheets

# CSS: Selectors / Properties / Values

```
1  /* Built-in selectors: the tags you use in your HTML (body, h1, p, a, img...) */
2  body {
3      background-color: #f0f0f0;
4      font-family: Arial, sans-serif;
5      margin: 0;
6      padding: 20px;
7  }
8  /* Class (custom) selectors: create your own with a period + name */
9  .highlight {
10     background-color: yellow;
11     font-weight: bold;
12 }
```

```
1 <p class="highlight">This text will be highlighted.</p>
```



*There are A LOT of selectors and properties — Google and AI are your friend for finding exactly what you want!*

[jimna-h.github.io/james\\_super\\_cool\\_is\\_201\\_cheatsheet](https://jimna-h.github.io/james_super_cool_is_201_cheatsheet)



IS 201 TA Lab